Profiling Bayesian Optimization Frameworks for Synthetic and ML Tasks: A Baseline Study

Meron Oumer, Anjia Lu, Nargiz Akhmetova
High Performance Computing Lab - Smith College

September 30, 2026

Abstract

This paper presents a comprehensive profiling study of four optimization frameworks — BoTorch, DeepHyper , Optuna, Random Search and Grid Search — evaluated across synthetic benchmark functions and real-world machine learning tasks. We measure the wall-clock time per iteration, Gaussian process fit latency, acquisition function optimization cost, and end-to-end time-to-solution. This baseline study will serve as the foundation for future work .

1 Introduction

Bayesian Optimization (BO) is a widely used strategy for optimizing expensive black-box functions, particularly in machine learning and scientific computing. By combining surrogate modeling with acquisition-based decision-making, BO aims to reduce the number of costly objective evaluations required to identify high-quality solutions. However, as BO has been increasingly deployed in high-performance computing (HPC) environments, its own internal computational overhead has emerged as a critical limitation.

In many modern workloads, the cost of BO components—such as Gaussian Process (GP) fitting, acquisition function optimization, and trial orchestration—can rival or even exceed the cost of evaluating the objective function itself. This issue is especially pronounced in low-latency settings, high-throughput tuning pipelines, and large-scale distributed systems, where sequential surrogate updates and synchronization barriers can lead to idle compute resources and poor scalability. Without careful analysis, attempts to accelerate BO risk optimizing the wrong components or introducing additional complexity without measurable performance gains.

In this work, we conduct a comprehensive profiling study of several widely used Bayesian Optimization frameworks, including BoTorch, DeepHyper, Optuna, and Ray Tune, alongside Grid Search and Random Search baselines. We evaluate these frameworks across a diverse set of workloads, ranging from synthetic benchmark functions (Branin, Hartmann, Rastrigin) to classical machine learning models (logistic regression, random forests, XGBoost) and large pretrained neural networks (DistilBERT,T5-Large). By instrumenting each framework with fine-grained timing hooks, we capture detailed per-iteration breakdowns of surrogate fitting, acquisition optimization, and objective evaluation.

Our primary contribution is a structured, empirical analysis of Bayesian Optimization performance in HPC environments. We provide framework-agnostic runtime decompositions that expose hidden bottlenecks, characterize how different workloads stress BO components, and establish a reproducible baseline for fair comparison across systems. These insights directly inform the design of more scalable BO implementations and serve as the foundation for future work on targeted performance optimizations for HPC-driven machine learning workflows

2 Literature Review

2.1 Bayesian Optimization FrameWorks

Bayesian Optimization (BO) relies on software frameworks to manage complex components such as surrogate modeling, acquisition function optimization, and trial scheduling. As BO has become widely adopted in machine learning workflows, several frameworks have emerged to support both research experimentation and large-scale deployment. However, these frameworks differ substantially in design philosophy, abstraction level, and support for parallel execution.

BoTorch is a modular, research-oriented framework built on PyTorch that focuses on efficient Monte Carlo Bayesian Optimization. By leveraging PyTorch's automatic differentiation and GPU acceleration, BoTorch enables flexible implementation of custom surrogate models and acquisition functions. Its low-level design makes it particularly suitable for algorithmic research, though this flexibility often comes at the cost of increased computational overhead and user-managed performance considerations. While higher-level interfaces such as Ax build on BoTorch to provide standardized experimentation pipelines, BoTorch itself exposes the full computational cost of GP inference and acquisition optimization to the user.

Optuna is a black-box optimization framework designed for ease of use and practical scalability. It supports a variety of search strategies, including random search, Bayesian optimization, and evolutionary methods, and incorporates aggressive early-stopping mechanisms such as Hyperband and asynchronous successive halving. Optuna's pruning strategies are particularly effective for reducing wasted computation on unpromising trials, but its internal profiling of BO components is limited, making it difficult to analyze fine-grained performance behavior.

DeepHyper is specifically designed for asynchronous hyperparameter optimization in high-performance computing environments. It emphasizes large-scale parallelism and is frequently applied to deep neural network workloads. By decoupling model evaluations from the optimization logic, DeepHyper enables efficient utilization of HPC resources, though its design introduces trade-offs in synchronization and scheduling overhead that warrant detailed performance analysis.

2.2 Bayesian Optimization in High-Performance Computing

In HPC environments, objective functions such as application runtime, energy consumption, or large-scale model training are often extremely expensive to evaluate. The configuration spaces involved—spanning compiler flags, runtime parameters, architectural choices, and algorithmic hyperparameters—are typically high-dimensional and combinatorial, making exhaustive search infeasible.

Model-based optimization methods, particularly Bayesian Optimization, have therefore been widely adopted in HPC tuning workflows. Frameworks such as HiPerBOt demonstrate how BO can efficiently identify high-performing configurations using a limited number of expensive evaluations. Importantly, BO in HPC settings often prioritizes identifying optimal configurations rather than modeling the objective function globally, allowing surrogate models to focus on informative regions of the search space.

A key advancement in HPC-oriented BO is the incorporation of parallel and asynchronous evaluation strategies. Since evaluations may take minutes or hours, parallelism is essential for practical efficiency. Extensions to classical acquisition functions—such as batch Expected Improvement—enable multiple configurations to be evaluated simultaneously. Furthermore, transfer learning techniques have been used to incorporate information from small-scale experiments into larger deployments, reducing overall optimization cost.

Despite these advances, parallelizing BO introduces new challenges. Communication overhead, synchronization barriers, and resource contention can negate theoretical speedups. Moreover, components such as GP inference may remain sequential or memory-bound, limiting scalability even when objective evalua-

tions are parallelized.

2.3 Profiling and Analysis in Machine Learning Systems

At the core of Bayesian Optimization is the interaction between a surrogate model and an acquisition function. Gaussian Processes (GPs) are commonly used due to their expressive uncertainty estimates and closed-form inference. Acquisition functions such as Expected Improvement (EI), Upper Confidence Bound (UCB), and Knowledge Gradient (KG) leverage these uncertainty estimates to balance exploration and exploitation.

However, the computational cost of these components varies significantly depending on problem size, dimensionality, and implementation details. GP inference scales cubically with the number of observations in its naïve form, motivating the development of scalable approximations such as Structured Kernel Interpolation (SKI). The KISS-GP variant of SKI significantly reduces computational cost by interpolating kernel evaluations onto inducing point grids, though it introduces new complexity challenges in higher dimensions.

Alternative surrogate models—including Bayesian Neural Networks (BNNs) and Student-t Processes (TPs)—have been explored to address limitations of GPs in high-dimensional or non-stationary settings. While these models offer increased flexibility, they often introduce additional computational overhead and implementation complexity, making their performance characteristics highly problem-dependent.

Despite extensive algorithmic research, relatively little work focuses on fine-grained profiling of BO systems. Most evaluations emphasize optimization performance (e.g., convergence speed or final objective value) rather than computational breakdowns. As a result, it remains unclear how time is distributed among GP fitting, acquisition optimization, and objective evaluation across different frameworks and workloads.

2.4 Gaps in the Literature

Several gaps motivate the need for systematic profiling of Bayesian Optimization frameworks:

- **Limited Runtime Decomposition**: Most BO studies report convergence metrics without analyzing internal time allocation, obscuring performance bottlenecks.
- **Scalability Beyond Low Dimensions:** BO methods typically perform best in low-dimensional settings ($d \leq 20$), and existing scalability techniques such as SKI still face exponential growth challenges.
- **Interaction of System-Level Bottlenecks**: Real-world workloads often combine high dimensionality, expensive evaluations, and parallel execution, yet few studies examine how these factors interact at the systems level.
- **Framework-Level Comparisons:** There is a lack of fair, reproducible comparisons across BO frameworks using consistent workloads and profiling methodology.

Framework-Level Comparisons: There is a lack of fair, reproducible comparisons across BO frameworks using consistent workloads and profiling methodology.

This work addresses these gaps by conducting a comprehensive, framework-agnostic profiling study of Bayesian Optimization systems across diverse workloads and HPC execution settings, establishing a foundation for principled performance optimization

3 Methodology

This section describes the experimental design, benchmark suite, profiling instrumentation, and evaluation metrics used to analyze the performance characteristics of Bayesian Optimization (BO) frameworks in high-

performance computing (HPC) environments. Our methodology is designed to isolate and measure the computational costs of individual BO components across a diverse set of workloads and execution settings.

3.1 Frameworks Profiled

We evaluate a representative set of widely used Bayesian Optimization and hyperparameter tuning frameworks that span both research-oriented and production-scale use cases:

- BoTorch — a modular, research-focused BO framework built on PyTorch, offering fine-grained control over surrogate models, acquisition functions, and optimization procedures.
- DeepHyper — an asynchronous hyperparameter optimization framework designed for scalable deep learning workloads in distributed environments.
- Optuna — a black-box optimization framework emphasizing ease of use, distributed execution, and aggressive early-stopping via pruning.
- Baselines — Grid Search and Random Search are included as non-adaptive baselines to contextualize BO overhead relative to simpler search strategies.

Each framework is used according to recommended best practices, with default configurations unless otherwise specified, to reflect realistic usage scenarios.

3.2 Synthetic Functions

Synthetic functions provide controlled settings where objective evaluation cost is low and BO overhead dominates runtime. We include:

- Hartmann 6D
- Branin
- Rastrigin

3.3 ML Tasks

Machine learning workloads introduce expensive and heterogeneous objective evaluations, reflecting realistic BO use cases:

- Logistic Regression
- Random Forest and XGBoost
- HuggingFace model fine-tuning (DisTillBert and T-5 Large)

3.4 Experiment Grid

3.5 Baseline Grid Search and Random Search Evaluation

To contextualize the performance of Bayesian Optimization frameworks, we implement **Grid Search** and **Random Search** as model-free baseline optimization strategies across synthetic benchmark functions and classical ML models. These baselines provide a lower-bound reference for runtime and convergence, enabling direct comparison of BO component overhead.

Table 1: Framework × Task Combinations

Framework	Synthetic Functions	Classical ML Models	Large Neural Models
BoTorch	✓	✓	✓
DeepHyper	✓	✓	✓
Ray Tune	✓	✓	✓
Optuna	✓	✓	✓
Random Search	✓	✓	✓
Grid Search	✓	✓	✓

3.5.1 Grid Search

Description: Grid Search exhaustively enumerates all combinations of hyperparameter values in a predefined discrete grid.

Procedure:

- For each task, we define grids over relevant hyperparameters (e.g., learning rates, regularization strengths, batch sizes).
- Each combination is evaluated using the task-specific objective function (e.g., cross-validated ROC-AUC for ML models or function value for synthetic functions).
- Iteration-wise evaluation times and best-so-far objective values are recorded to facilitate convergence analysis.

Metrics Collected:

- `eval_time`: Wall-clock time to evaluate the objective function for a single hyperparameter combination.
- `total_iter_time`: Time for candidate generation and evaluation for each iteration.
- `best_y`: Best observed objective value up to that iteration.

Note: Grid Search does not involve surrogate modeling, so `gp_fit_time` and `acq_opt_time` are set to zero.

3.5.2 Random Search

Description: Random Search samples hyperparameters uniformly from the unit hypercube, offering a stochastic but computationally cheaper baseline.

Procedure:

- Candidate hyperparameters are generated by mapping uniform unit-hypercube samples to task-specific ranges.
- Each candidate is evaluated using the same objective function as Grid Search.
- Iteration-wise performance and cumulative best values are recorded.
- Profiling hooks for acquisition generation (`acq_opt_time`) are included to maintain consistency with Bayesian Optimization frameworks, although surrogate fitting is not performed (`gp_fit_time` = 0).

Metrics Collected:

- `gp_fit_time`: 0 (not applicable)
- `acq_opt_time`: Time to generate candidate samples
- `eval_time`: Wall-clock evaluation time
- `total_iter_time`: Sum of candidate generation and evaluation times
- `best_y`: Cumulative best objective value

3.5.3 Time-to-Quality Analysis

For both baseline strategies, we track best-so-far objective performance versus cumulative wall-clock time to allow direct comparison with BO frameworks. Optional Chrome profiler traces provide detailed CPU activity breakdowns for iteration-level analysis.

3.5.4 Applicability

Grid and Random Search are applied to:

- Synthetic benchmark functions: Branin, Hartmann, Rastrigin
- Classical ML tasks: Logistic Regression, Random Forest, XGBoost

They are not applied to large-scale transformer fine-tuning tasks (DistilBERT, T5-Large) due to prohibitive evaluation costs.

3.6 HardWare and Experiment Excecution Environment

- **CPU**: Multi-core x86 processors
- **Memory**: 50 GB RAM per node
- **Software Stack**: Python, PyTorch, Framework-specific dependencies

3.7 Metrics Collected

- GP fit time: Time spent fitting or updating our surrogate model.
- Acquisition function time : Time spent optimizing the acquisition function
- Evaluation time : Time spent evaluating the objective function
- Total iteration time : End-to-end time per BO iteration

3.8 Reproducbility

All experiments are conducted using fixed random seeds, consistent search budgets, and version-locked dependencies. Logging scripts and profiling utilities are designed to be framework-agnostic, enabling reproducibility and extension to additional BO systems in future work.

4 Results

This section presents a comprehensive analysis of the profiling experiments across synthetic benchmark functions and ML workloads. Stacked bar charts illustrate the iteration-wise breakdown of wall-clock time into GP fit time, acquisition optimization time, and evaluation time, allowing a fine-grained examination of framework-specific bottlenecks and scalability trends.

4.1 Time Contributions

4.1.1 Botorch

- For synthetic functions such as Hartmann6, GP fit time dominates iteration cost, reaching spikes of ≈ 3.75 seconds.
- In ML tasks, such as Logistic Regression and XGBoost, acquisition optimization time often rivals GP fitting, occasionally exceeding it in specific trials.
- Increasing dimensionality (Branin $\rightarrow$ Hartmann6) and the number of observations leads to higher GP fit time, reflecting the $O(N^3)$ scaling of the Gaussian Process.

4.1.2 DeepHyper

- Surrogate fit time is consistently near-zero across all tasks ($<$0.2ms), while candidate selection time dominates iteration cost.
- Even for large-scale neural models like DistilBERT or T5-Large, DeepHyper overhead remains negligible (< 0.25ms), demonstrating its efficiency for HPC workloads.

4.1.3 Optuna

- GP fit time dominates the first trial (Trial 0), causing spikes of up to 6 seconds for Branin and 2.4 seconds for Rastrigin.
- For large models (T5-Large), acquisition optimization becomes the main bottleneck in some trials, exceeding 400ms.

We can see that the synthetic tasks highlight the hidden BO overheads, while the more expensive ML tasks shift the dominant time to object evaluation.

Framework choice has a dramatic effect on iteration time: BoTorch and Optuna operate in seconds, whereas DeepHyper operates in milliseconds.

4.2 FrameWork Comparisons

Total Iteration Time

- DeepHyper consistently exhibits the lowest total iteration time across all tasks.
- BoTorch and Optuna scale poorly with task complexity; T5-Large induces fit-time spikes exceeding 5 seconds.
- Surprisingly, even simple tasks (Branin) result in BoTorch iterations exceeding 2 seconds, demonstrating the impact of GP overhead in low-cost objective scenarios.

Scalabiility

- DeepHyper handles complex tasks with negligible impact on iteration time.
- BoTorch and Optuna show significant growth in GP fit time as the number of trials and observations increases.

4.3 Task Specific Behaviour

Synthetic vs ML tasks

- Synthetic functions (Branin, Rastrigin, Hartmann6) reveal that GP fit time and acquisition optimization are significant bottlenecks in BoTorch and Optuna.
- In ML tasks with costly evaluations (XGBoost, T5-Large), BO overhead is less impactful, though cumulative fit time can still reach tens of seconds (e.g., 42 seconds for XGBoost in BoTorch by iteration 20).

Large Scale Neural Models

- BoTorch exhibits volatile GP fit times for T5-Large (Trial $22 > 5,000$ ms), indicating that the surrogate model continues to strain under large observation sets.
- DeepHyper maintains near-constant iteration cost, highlighting the advantage of lightweight surrogate approximations for HPC deployment.

5 Discussion

The profiling results provide a detailed understanding of the computational behavior of Bayesian Optimization frameworks across synthetic functions, classical ML models, and large-scale neural networks. Several key insights emerge regarding framework efficiency, scalability, and suitability for HPC deployment.

5.0.1 Framework-Level Insights

BoTorch and Optuna:

- Both frameworks exhibit significant overhead due to GP fitting and acquisition optimization, particularly as the number of observations increases.
- Sequential GP updates introduce bottlenecks that amplify with iterations, dimensionality, and task complexity.
- While these frameworks offer algorithmic sophistication and theoretically efficient search, the overhead may outweigh benefits in low-latency or high-throughput environments.
- Unexpected spikes (e.g., Trial 0 in Optuna, Trial 22 in BoTorch) suggest that initialization or re-fitting operations can temporarily freeze worker nodes, further limiting parallel efficiency.

DeepHyper:

- Demonstrates minimal internal overhead, with surrogate fit times consistently near-zero and candidate selection times in the sub-millisecond range.

- Maintains near-constant iteration cost across all task types and scales, making it highly suitable for HPC environments where parallel evaluation efficiency is critical.
- Its lightweight surrogate approach highlights the potential of SKI-like or approximate models to bypass the $O(N^3)$ cost of standard Gaussian Process fitting.

Baselines (Grid and Random Search):

- Perform extremely well for low-cost synthetic functions due to negligible internal overhead.
- Time-to-quality is superior to BO frameworks for small-scale tasks but loses advantage as objective evaluation cost increases (e.g., XGBoost or large neural models).
- Serve as an important reference for assessing whether the computational complexity of a BO framework is justified by its performance gains.

5.0.2 Task-Specific Observations

Synthetic Functions:

- Reveal hidden computational costs of BO frameworks. GP fitting and acquisition optimization dominate iteration time in BoTorch and Optuna.
- Grid and Random Search often outperform BO in time-to-quality for simple low-dimensional tasks, highlighting the trade-off between "smart" search and internal overhead.

Classical ML Models:

- Expensive objective evaluations reduce the relative impact of BO overhead.
- Cumulative GP fit time in BoTorch can reach tens of seconds, but the overall reduction in required function evaluations can still make BO advantageous.

Large Neural Models (T5-Large, DistilBERT):

- DeepHyper remains highly efficient, demonstrating that lightweight surrogate approaches scale well to high-dimensional and expensive tasks.
- BoTorch and Optuna exhibit volatile and growing overhead, indicating that sequential GP fitting and acquisition optimization may hinder scalability in distributed HPC setups.

5.0.3 Scaling and Parallelization Implications

- **Sequential Bottlenecks:** BoTorch and Optuna are vulnerable to idle worker time in distributed environments, as master nodes handle expensive GP fitting sequentially.
- **Parallel Efficiency:** DeepHyper's negligible surrogate overhead ensures near-optimal utilization of HPC nodes, making it well-suited for asynchronous and batch evaluation scenarios.
- **Dimensionality Sensitivity:** GP fitting in BoTorch scales poorly with problem dimensionality, whereas DeepHyper remains mostly insensitive, even in 6D synthetic tasks or high-dimensional hyperparameter spaces.

5.0.4 Practical Recommendations

- **For HPC Deployments:** Lightweight surrogate models, as exemplified by DeepHyper, enable low-overhead, parallel-friendly BO that minimizes worker idling.
- **For Low-Cost Tasks:** Grid or Random Search may outperform BO frameworks in time-to-quality due to minimal iteration overhead.
- **For High-Dimensional or Expensive Tasks:** BO frameworks remain beneficial if the cost of objective evaluation dominates internal overhead. However, mitigating GP fitting costs—via SKI approximations or alternative surrogate models—is critical for scaling.
- **Monitoring Initialization Spikes:** Framework initialization spikes (Trial 0) should be anticipated and mitigated, especially in parallel execution contexts.

6 Conclusion

This study provides a framework-agnostic, fine-grained profiling of Bayesian Optimization systems across synthetic functions, classical ML models, and large-scale neural networks. We show that overheads vary dramatically by framework: BoTorch and Optuna incur significant GP fitting and acquisition optimization costs, while DeepHyper remains near-instantaneous even on complex tasks. Baselines like Grid and Random Search highlight that for low-cost objectives, simpler approaches can outperform BO in time-to-quality. Overall, these results offer a reproducible baseline and actionable insights for optimizing BO in HPC environments.

7 Future Work

Future directions could multiple options:

Reduce distributed synchronization overhead

Investigate zero-copy communication approaches (e.g., RDMA-style state sharing) to avoid expensive serialization and data transfer costs observed in distributed BoTorch and Optuna runs.

Lower internal BO computation costs

Explore compile-time optimizations such as kernel fusion or specialized code generation to reduce the cost of GP fitting and acquisition optimization, which dominate iteration time in Python-based frameworks.

Systems-level framework design

Study alternative implementations using low-overhead systems languages with strong compile-time guarantees to balance algorithmic flexibility with the performance and scalability needed for HPC environments.

References

1. Brochu, E., Cora, V., & de Freitas, N. *A Tutorial on Bayesian Optimization of Expensive Cost Functions*. 2010.
2. Shahriari, B., Swersky, K., Wang, Z., Adams, R. P., & de Freitas, N. *Taking the Human Out of the Loop: A Review of Bayesian Optimization*. Proceedings of the IEEE, 2016.

3. Balaprakash, P., Salim, M., Uram, T. D., Vishwanath, V., & Wild, S. M. *DeepHyper: Asynchronous Hyperparameter Search for Deep Neural Networks*. IEEE HiPC, 2018.

4. Menon, H., et al. *Auto-tuning Parameter Choices in HPC Applications using Bayesian Optimization*. IEEE IPDPS, 2020.

5. Feurer, M., Klein, A., Eggensperger, K., Springenberg, J., Blum, M., & Hutter, F. *Auto-sklearn: Efficient and Robust Automated Machine Learning*. Springer, 2019.

6. Wilson, A. G., & Nickisch, H. *Kernel Interpolation for Scalable Structured Gaussian Processes (KISS-GP)*. ICML, 2015.

7. Yadav, A., et al. *Kernel Interpolation with Sparse Grids*. NeurIPS, 2022.

8. BoTorch, Ray Tune, Optuna, Hyperopt. *Official GitHub repositories and documentation*.

9. PyTorch Profiler Documentation. *Performance Tuning Guide*.

10. Additional resources on scalable Gaussian Processes, sparse GP approximations, and distributed Bayesian optimization (arXiv / NeurIPS proceedings).